By default, Active Record doesn't know about the connection between these associations. This can lead to two copies of an object getting out of sync:

```
c = Customer.first
o = c.orders.first
c.first_name == o.customer.first_name # => true
c.first_name = 'Manny'
c.first_name == o.customer.first_name # => false
```

This happens because `c` and `o.customer` are two different in-memory representations of the same data, and neither one is automatically refreshed from changes to the other. Active Record provides the `:inverse_of` option so that you can inform it of these relations:

```
class Customer < ActiveRecord::Base
  has_many :orders, inverse_of: :customer
end

class Order < ActiveRecord::Base
  belongs_to :customer, inverse_of: :orders
end
```

With these changes, Active Record will only load one copy of the customer object, preventing inconsistencies and making your application more efficient:

```
c = Customer.first
o = c.orders.first
c.first_name == o.customer.first_name # => true
c.first_name = 'Manny'
c.first_name == o.customer.first_name # => true
```

There are a few limitations to `inverse_of` support:

- They do not work with `:through` associations.
- They do not work with `:polymorphic` associations.
- They do not work with `:as` associations.
- For `belongs_to` associations, `has_many` inverse associations are ignored.

Every association will attempt to automatically find the inverse association and set the `:inverse_of`option heuristically (based on the association name). Most associations with standard names will be supported. However, associations that contain the following options will not have their inverses set automatically:

- `:conditions`
- `:through`
- `:polymorphic`
- `:foreign_key`

4 Detailed Association Reference

The following sections give the details of each type of association, including the methods that they add and the options that you can use when declaring an association.

4.1 `belongs_to` Association Reference

The `belongs_to` association creates a one-to-one match with another model. In database terms, this association says that this class contains the foreign key. If the other class contains the foreign key, then you should use `has_one` instead.

4.1.1 Methods Added by `belongs_to`

When you declare a `belongs_to` association, the declaring class automatically gains five methods related to the association:

- `association(force_reload = false)`
- `association=(associate)`